



# House Price Prediction

Proyecto de Machine Learning

**Artificial Intelligence Foundations — Fundació URV**

Adrià · Raul | Mayo 2026



# ¿Qué queríamos conseguir?

## El problema

Predecir el precio de una casa (SalePrice)  
a partir de sus características  
(tamaño, calidad, ubicación...)

## El dataset

Kaggle House Prices  
1.460 casas · 80 variables  
Datos reales de Ames, Iowa

## El enfoque

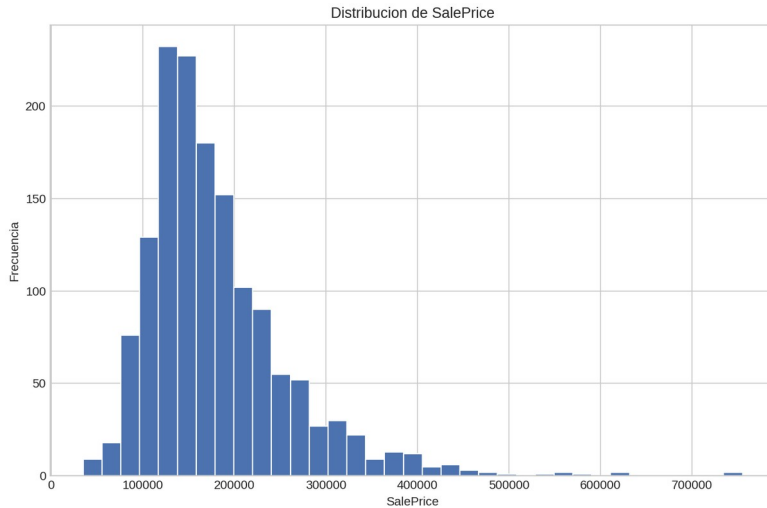
Pipeline completo de ML:  
EDA → Preprocesamiento → Modelo → App

## El resultado

App interactiva en Streamlit  
donde puedes meter datos  
y obtener una predicción

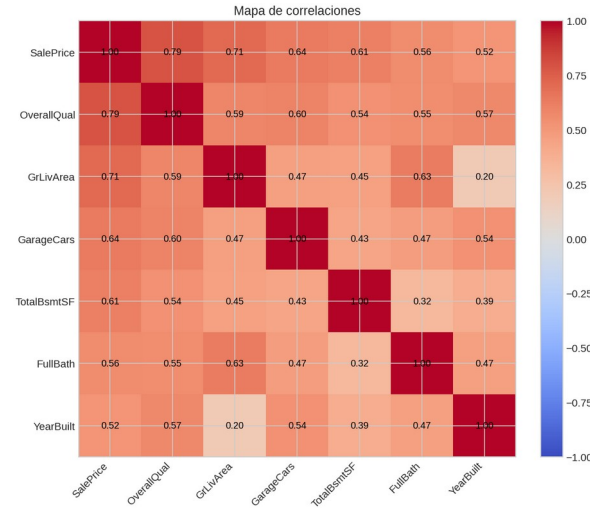


# Explorando los datos (Exploratory Data Analysis)



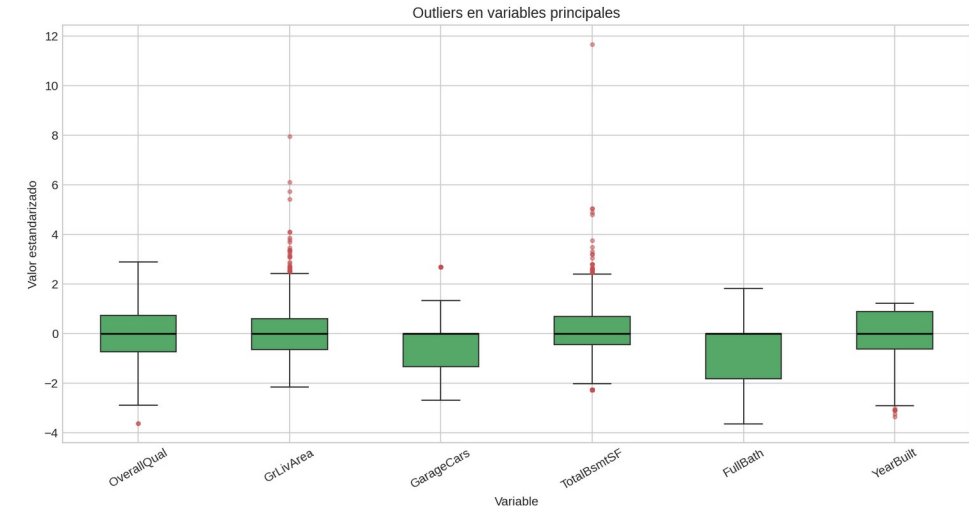
## Precios de las casas

Mayoría entre 100k y 200k  
Distribución asimétrica  
(la mayoría son casas asequibles)



## ¿Qué afecta al precio?

OverallQual (0.79) ← calidad  
GrLivArea (0.71) ← metros  
GarageCars (0.64) ← garaje



## Valores faltantes

Hay columnas con muchos valores vacíos, como PoolQC (99.5%) o MiscFeature (96.3%)



## Herramientas:

Hemos usado pandas (datos), seaborn (generación gráficos) y matplotlib (exportar gráficos) para entender la distribución del precio, detectar nulos/outliers y elegir las variables más relacionadas con SalePrice.



# Limpieza y preparación de los datos (scikit-learn )

## Rellenar huecos

Valores vacíos:

- Números → mediana (para evitar la desviación de la media con valores extremos)
- Categorías → moda (el valor más frecuente de los textos existentes)

## Estandarizar

Poner todos los números en la misma escala con la clase `StandardScaler` de la librería `scikit-learn` para que el modelo no vea valores grandes en los m2 y valores pequeños en el número de habitaciones, p.ej.

## Convertir texto a números

Categorías → con la clase `OneHotEncoder`, cada opción (texto) se vuelve una columna de 0/1.



## División del dataset:

### 80% Train

1.168 casas  
para entrenar

### 20% Test

292 casas  
para evaluar

¿Por qué separar?

Si evaluamos con los mismos datos que usamos para entrenar, el modelo haría "trampa" y no sabríamos si funciona de verdad con casos nuevos.



# Las 6 variables más importantes

Nos quedamos con las que más influyen en el precio. Así el modelo es más sencillo de entender.

**OverallQual**

0.79

Calidad de la casa

**GrLivArea**

0.71

Metros cuadrados habitables

**GarageCars**

0.64

Plazas de garaje

**TotalBsmtSF**

0.61

Tamaño del sótano

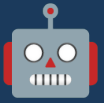
**FullBath**

0.56

Baños completos



Con solo 6 variables podemos predecir el precio con bastante precisión. Menos es más.



# Primer modelo

Empezamos con un modelo sencillo: Random Forest — sin ajustar, con los valores por defecto.



## Configuración

Algoritmo: RandomForestRegressor

300 árboles de decisión

Entrenado con las 6 variables



## Resultados en test

RMSE

**\$29.102**

Error típico (en \$)

MAE

**\$19.067**

Error absoluto medio

$R^2$

**0.8896**

Precisión (1 = perfecto)



## ¿Qué significa $R^2 = 0.89$ ?

El modelo explica el 89% de la variación de los precios. Dicho de otro modo: de cada 10 casas, el modelo acierta la tendencia de casi 9.

El error típico es de unos \$29.000 — que para ser el primer modelo con solo 6 variables, ¡no está nada mal!



# Intentando mejorar el modelo

Probamos a ajustar los parámetros del Random Forest para ver si podíamos mejorarlo.



## Cómo lo hicimos

Usamos los Jupiter Lab generados para modificar los parámetros:

- RandomizedSearchCV: prueba 25 combinaciones de 432 posibles
- 3 validaciones cruzadas por cada prueba  $\rightarrow 3 \times 25 = 75$  entrenamientos en total
- Parámetros usados: profundidad, número de árboles, muestras mínimas, etc.



## Resultados

RMSE: \$29.724 (antes \$29.102)

MAE: \$19.331 (antes \$19.067)

$R^2$ : 0.8848 (antes 0.8896)



**El modelo por defecto funcionó mejor**



## Conclusiones:

El modelo por defecto ya era muy bueno gracias a la selección correcta que hicimos de las 6 variables más representativas.



# Comparativa: Baseline vs Tuned

| Métrica        | Baseline | Tuned    | ¿Mejóro? |
|----------------|----------|----------|----------|
| RMSE           | \$29.102 | \$29.724 | ✗ No     |
| MAE            | \$19.067 | \$19.331 | ✗ No     |
| R <sup>2</sup> | 0.8896   | 0.8848   | ✗ No     |

**El modelo baseline con valores por defecto es suficiente y más robusto**

## Conclusión

Para este proyecto, con lo básico bien hecho obtenemos un resultado sólido.  
En el futuro podríamos probar otros modelos (XGBoost, Gradient Boosting) para intentar mejorarlo.





# La app en Streamlit



## Predictor de precios de vivienda

Aplicación desarrollada para AI Foundations — Fundació URV

Comparativa entre RandomForestRegressor baseline y modelo con tuning de hiperparámetros.

### Introduce las características de la vivienda

Calidad general (OverallQual)



Superficie habitable (m²)

140,00

Capacidad del garaje (GarageCars)



### Pruébalo tú mismo

Mueve los deslizadores de calidad, metros cuadrados, garaje... y el precio se actualiza automáticamente. Así de fácil.

Fork



### Características

Interfaz sencilla e intuitiva  
Ajusta las 6 variables clave  
Predicción al instante  
Sin instalar nada en tu PC



### Acceso

Disponible online en:  
[house-prices-v3nrejcyjltytdctev9hgk.streamlit.app](https://house-prices-v3nrejcyjltytdctev9hgk.streamlit.app)



# El pipeline completo



## EDA

Análisis de los datos



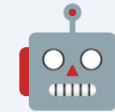
## Limpieza

Rellenar huecos y estandarizar



## Selección

Elegir las 6 mejores variables



## Modelo

Random Forest entrenado

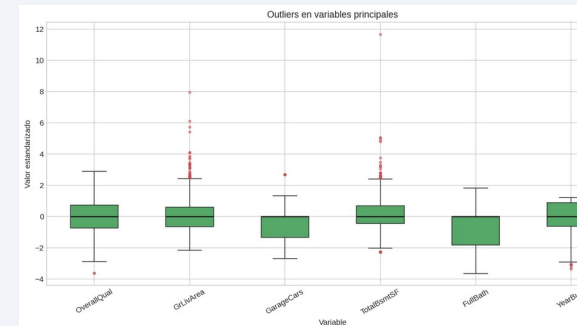
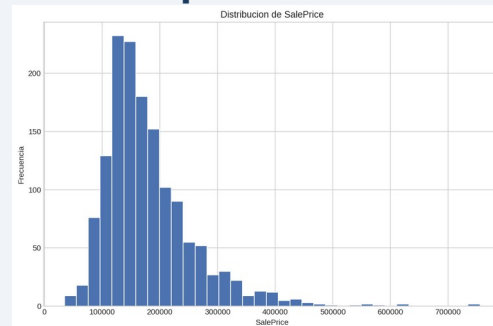
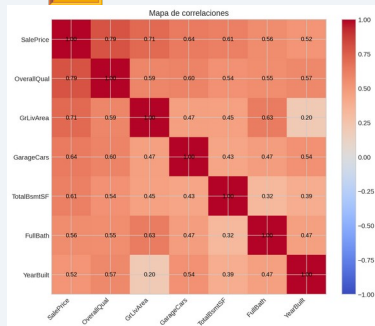


## App

Predicción interactiva



## Así se ven los datos en cada etapa:



## Predictor de precios de vivienda

Aplicación desarrollada para la AI Foundations - Fundación URV

Comparativa entre Random Forest y Gradient Boosting y modelo con testing de hiperparámetros.

Introduce las características de la vivienda

Calidad general (OverallQual)

Superficie habitable (m²)

Capacidad del garaje (GarageCars)

Superficie sótano (m²)

Baños completos (FullBath)

Año de construcción (YearBuilt)

2000



# Qué aprendimos con este proyecto

## ✓ **R<sup>2</sup> de 0.89**

Con solo 6 variables nuestro modelo acierta el 89% de la variación de precios

## ✓ **Pipeline completo**

Aprendimos a hacer todo el proceso: desde los datos hasta una app funcionando

## ✓ **Un buen preprocesado marca la diferencia**

Entender, limpiar y seleccionar los datos es tan importante como el modelo

## ✓ **La simplicidad también funciona**

A veces el modelo inicial ya es suficientemente bueno: más complejidad no siempre significa mejores resultados



## **Próximos pasos**

Probar XGBoost, Gradient Boosting, más variables, log-transform...



# ¡Gracias!



**Repositorio**

[github.com/Racap/house-prices](https://github.com/Racap/house-prices)



**App Streamlit**

[house-prices-v3nrejcyjlttdctev9hgk.streamlit.app](https://house-prices-v3nrejcyjlttdctev9hgk.streamlit.app)



**Vídeo demostración**

[youtu.be/KWWt\\_EOOM8A](https://youtu.be/KWWt_EOOM8A)



**Documentación técnica**

[doc.pdf](#)

¿Preguntas? 😊